SWEN30006
Software Modelling and Design

Subject Overview

Semester 1, 2025
Today's Lecturer: Dr Philip Dart
Subject Coordinator: Dr Philip Dart

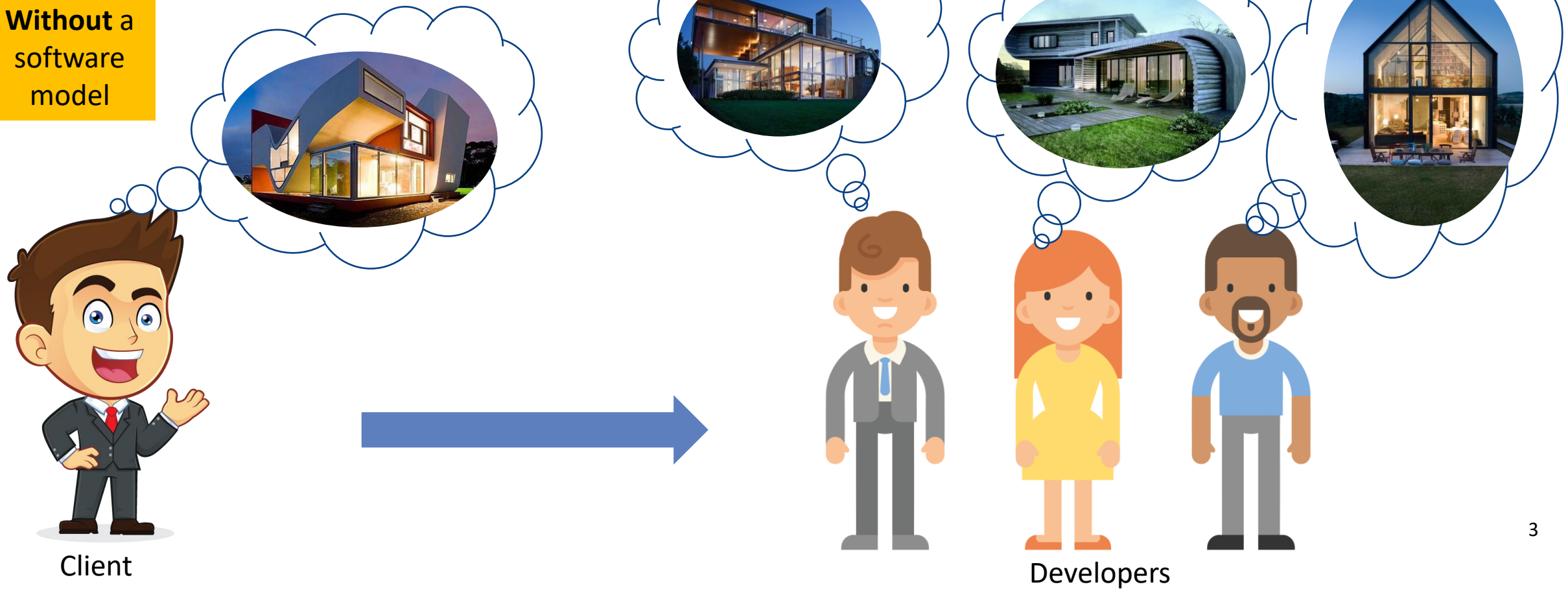
"An ugly system is one in which there are special interfaces for everything you want to do. Unix is the opposite. It gives you the building blocks that are sufficient for doing everything. That's what having a clean design is all about."

— Linus Torvalds, Just for Fun: The Story of an Accidental Revolutionary

Why software modelling is important?

- A software **model** helps communication. It can be used as a shared artifact to share understanding about the system.

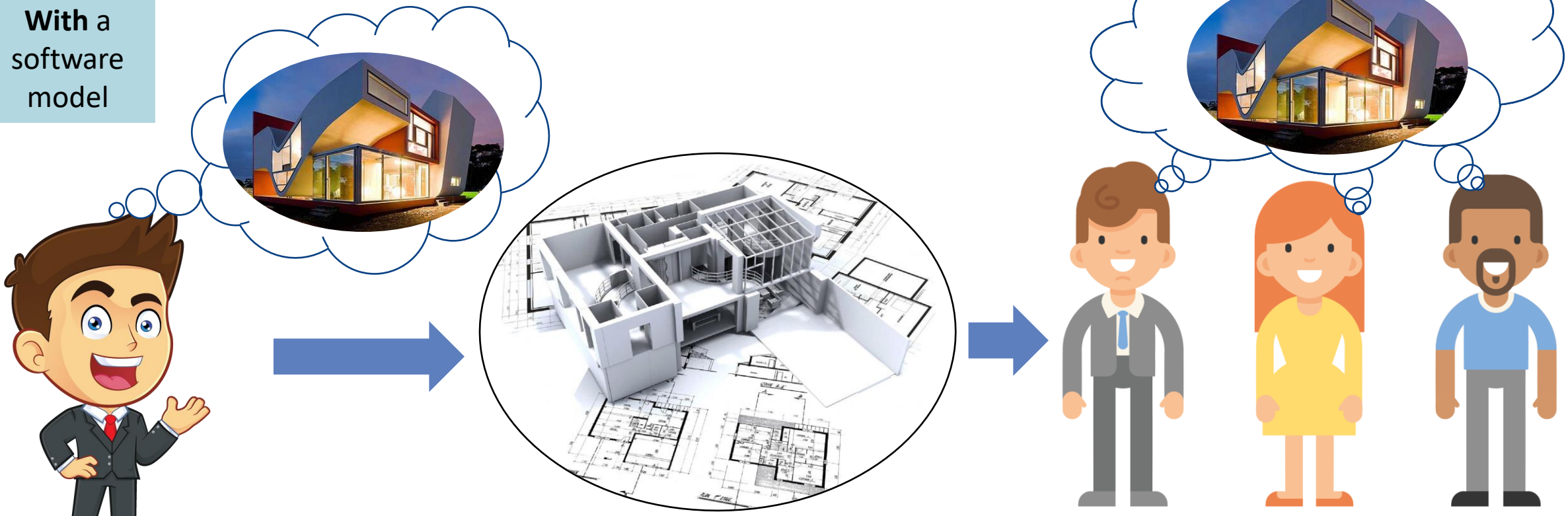
Without a software model



Why software modelling is important?

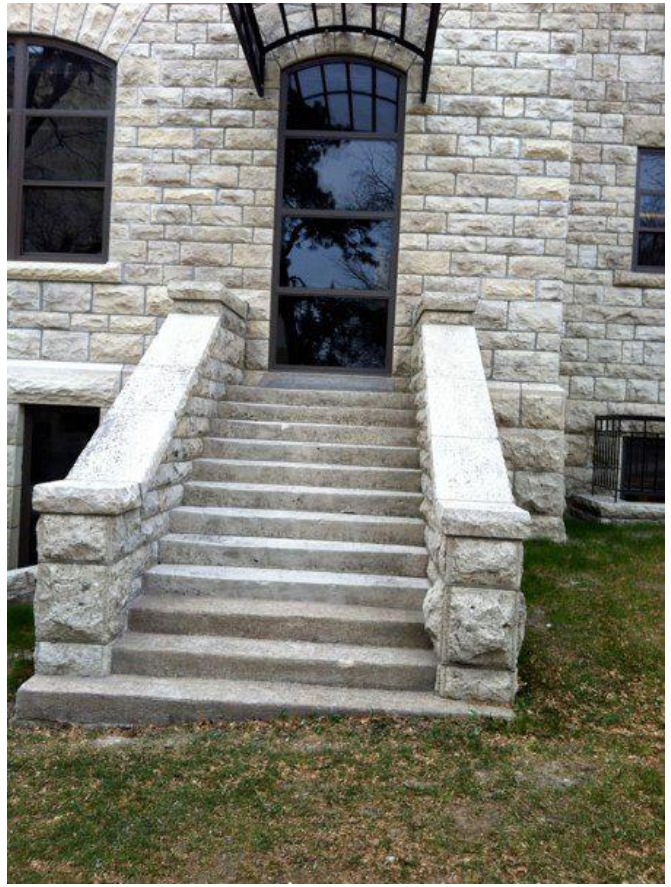
- A software **model** helps communication. It can be used as a shared artifact to share understanding about the system.

With a software model



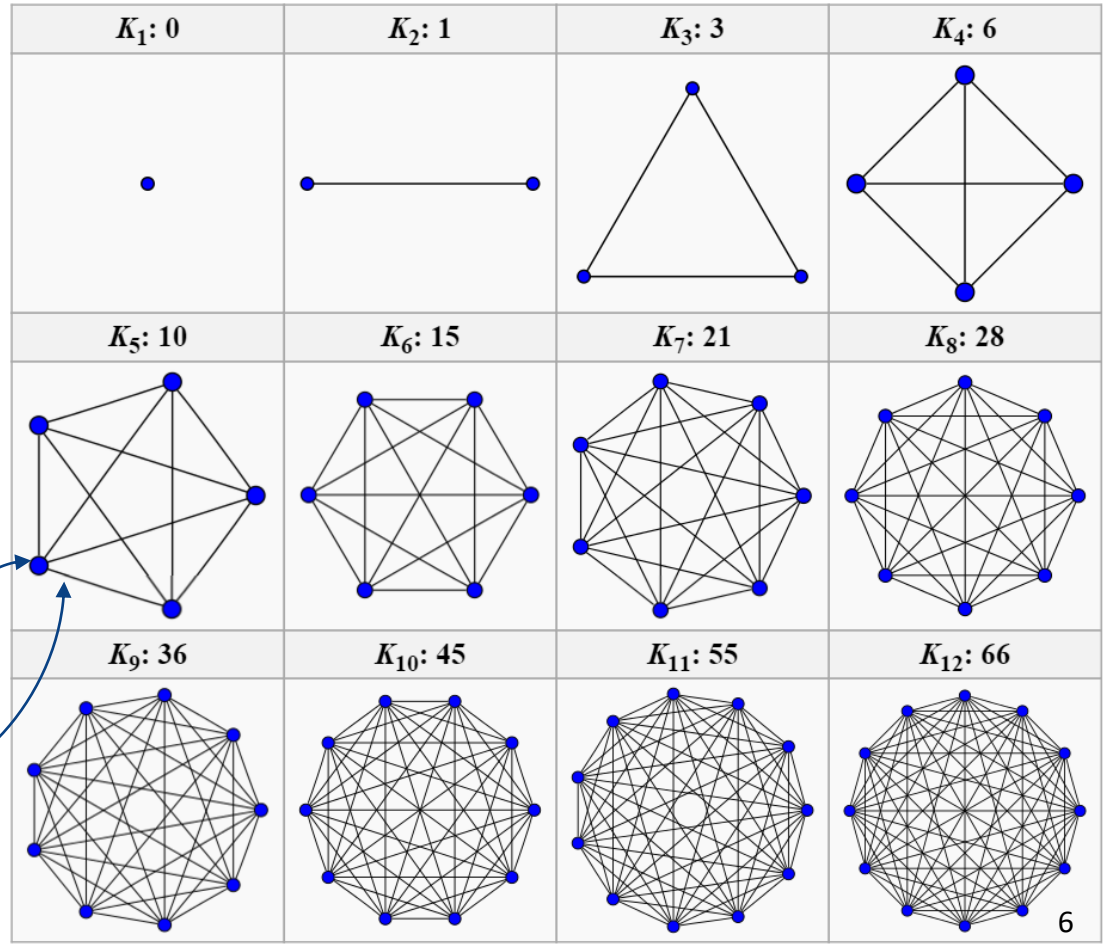
Why software design is important?

- Software **design** should improve simplicity and effectiveness of a software system as well as ease its maintainability



Why software design is important?

- Software **design** should improve simplicity and effectiveness of a software system as well as ease its maintainability
  - Software is abstract: relationships are not limited by physical proximity.
  - With some design:
    - All *N* components are interacting with each other
    - The complexity will be K_N: N*(N – 1)/2



A software component

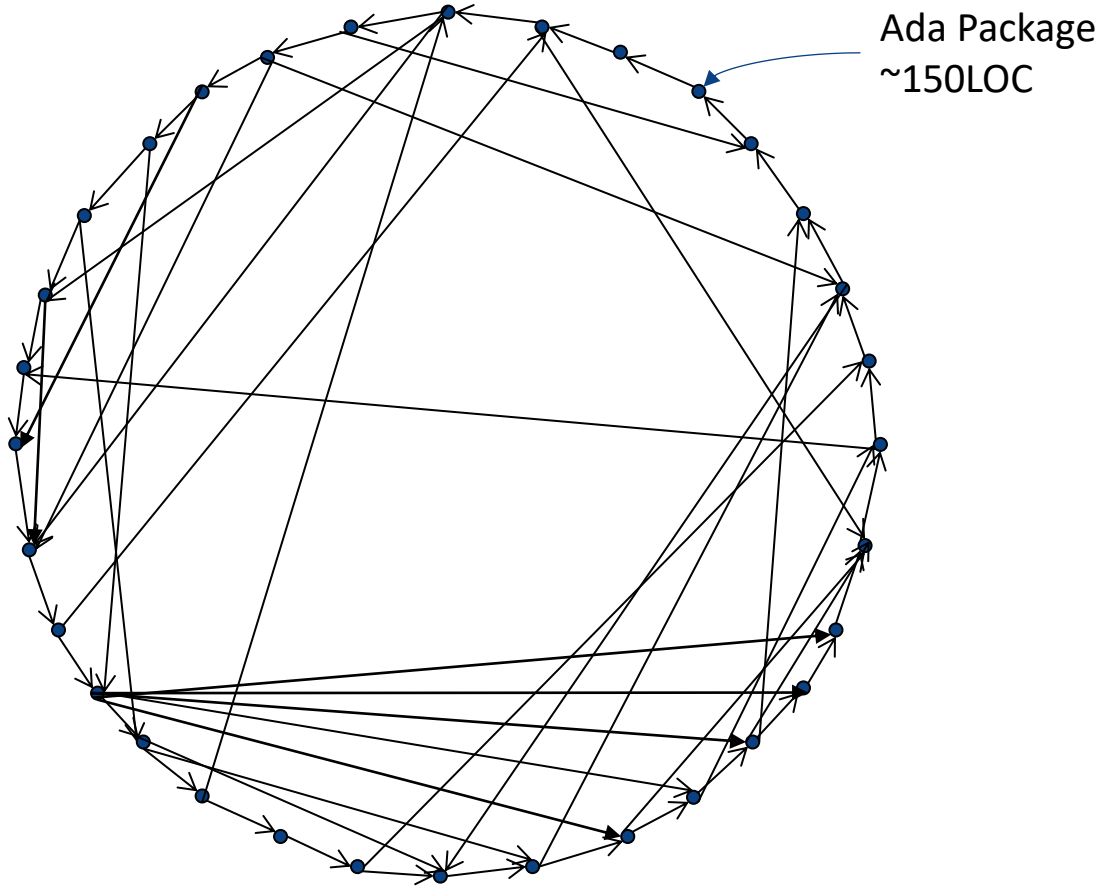
Dependency

https://en.wikipedia.org/wiki/Complete_graph

Example: Military Helicopter On-board Software

The software includes

- Navigation System
- Weapons Control System
- Communications System
- …
- Flight Control System (500 KLOC)
  - Build1 (150 KLOC)
    - Subsystem (15 KLOC - 10% of Build 1, 100 Ada packages)



Flight Control System
(Subsystem Build 1)

Each package in the cluster depends on every other package in the cluster.
High coupling → hard to understand → poor design

Lesson from the case study

- Software complexity is a problem in practice, not just in principle
- Software developers tend to add complexity unnecessarily if unconstrained
- Requiring developers to think through the implications of a structure/design change is important in maintaining a good design
- These issues are particularly critical in large projects

SWEN30006: Aims & Objectives

- The **aim** of the subject is to teach you about 'Software Modelling' and 'Software Design'
- Software Modelling is the creation of tangible, but abstract, representations of a system so that you can communicate your design ideas, critique them and explore viable alternatives
- Software Design is all about *purposefully* choosing the **behaviour** and **structure** and of your software system.
  - System behaviour: how your systems responds to inputs and events
  - System structure: how parts of the system collaborate to achieve the goals of the system

Software Modelling and Design subject

- The subject will focus on the object-oriented design method, with object-oriented modelling and modelling heuristics.
- UML (Unified Modelling Language) will be used as the primary modelling notation.
- Java will serve as the programming language to explore and validate the important design ideas.

Assumed Knowledge

- Object-Oriented Programming
- Algorithms and Data Structures
- Java Programming

Relationship with Other Subjects

- SWEN90009 Software Requirement and Analysis
- SWEN90007 Software Design and Architecture
- SWEN90004 Modelling Complex Software Systems

Case Study 1: Point-of-Sale System (from Larman's textbook)

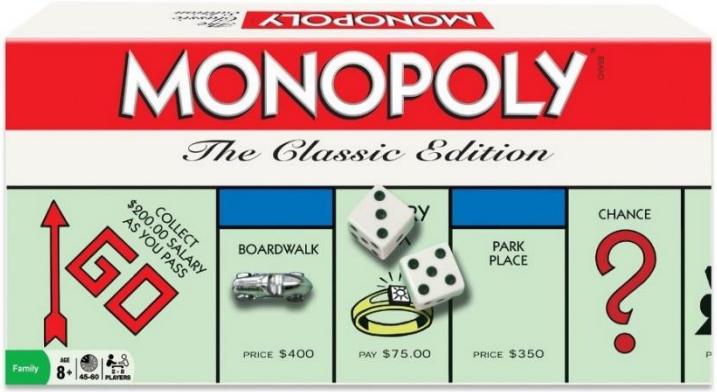
Point-Of-Sale (POS) is a system to record sales and handle payments.

Includes hardware such as a register with bar code scanner and credit card reader, as well as software.

Features include:

- Interfaces to service applications, e.g. tax calculator, inventory control
- Fault-tolerant: at least capture sales and handle cash payments
- Flexibility in client-side terminals and interfaces
- Able to support different clients with different business rules, e.g. discounting policies

Case Study 2: Monopoly Game System (from Larman's textbook)

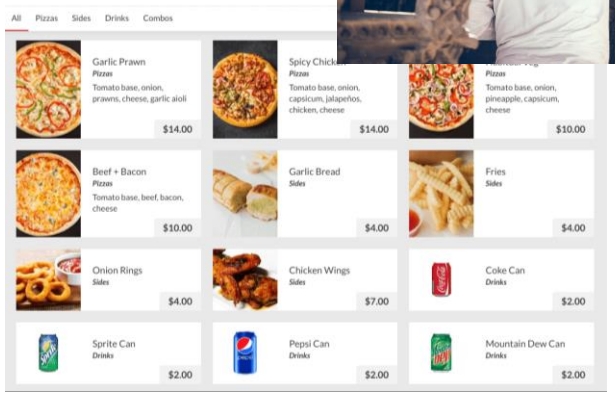


A Software Simulation of Monopoly

User starts off game and watches the activities of the simulated players.

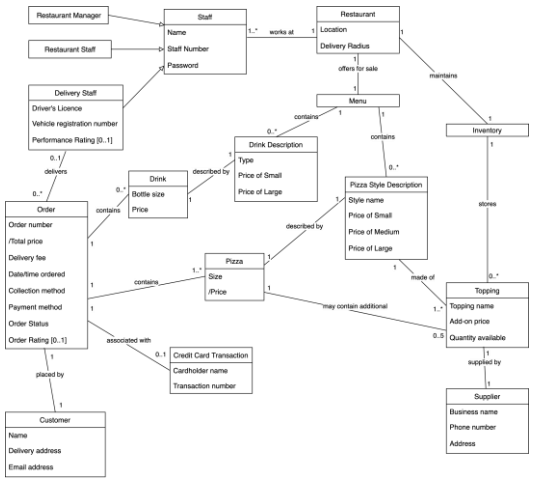
Case Study 3: Tony's Pizza System

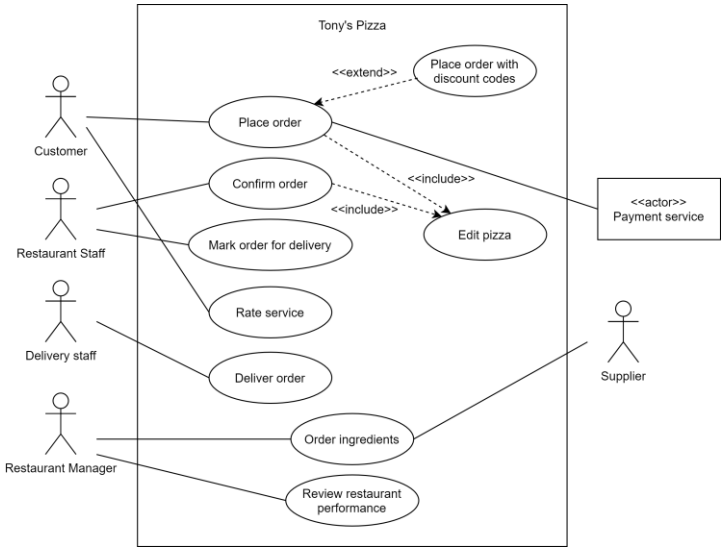
The complete guided case study (with examples of models and tutorials)



A system that will eventually encompass all facets of the business of Tony's pizza, at each location.

The system will be built incrementally, and we will be exploring the requirements of this system through software models and design principles throughout the semester.





Teaching & Learning

- **Lecture**
  - One 1-hour lecture per week
- **Pre-workshop learning** (self-study)
  - A 30mins video for supp. materials
  - Tony's pizza: A completed guided case study (with examples and tutorials)
- **Workshop**
  - One 2-hour workshop per week
  - Work on the exercises as a team
  - Exercises will correspond with the lecture content of the previous week
    - Workshop 1 on Week 2 will be related to the lecture content in Week 1
    - Workshop 0 on Week 1 is for class preparation and team forming

Reminder: Teams should be formed before Week 2

Assessment

- **8%** for Workshop **active** participation (Individual marks)
- **32%** for 2 Project assignments (Group-based marks)
- **60%** for Final examination
  - A 2-hour end-of-semester written examination

Hurdle

To pass the subject, students must obtain at least 50% overall, 20/40 in project + workshop, and 30/60 in the final exam

All the information and materials is available in the LMS

Teaching Team: Lecturers

Dr Philip Dart
(Subject Coordinator)

Dr Patanamon Thongtanunam

Dr Rajesh Chittor Sundaram

Contact details are available on the LMS > Teaching team & Contact Hours

Teaching Team: Tutors

Khang Vo
(Head Tutor)

Suijuan (Ashley) Wang

Guanqin (Harry) Wang

Zhengkang (Alex) Xiang

Wachiraphan (Ping) Charoenwet

Afsana Imam

Yun-Chi (Jim) Hsiao

Nadeeshan Dissanayake

Contact details are available on the LMS > Teaching team & Contact Hours

Contact & Support

Communication: LMS announcement, Emails, Ed Discussion

Consultation: Schedule appointment via email (15 - 30 mins based on request volume)

Topics	Contact
Personal issues related to the subject & assessments	• Email to the subject coordinator
Lecture content	• During the lecture class • Ed Discussion*
Workshop activities	• During the workshop class • Ed Discussion*
Assignments	• Ed Discussion*
Extension request	• Refer to the instructions in the assignment page
Exam	• Ed Discussion*

** One tutor and a corresponding lecturer will oversee inquiries on Ed Discussion. Expect replies within 3-5 business days. Feel free to follow up via email to a corresponding lecturer if the response takes longer than expected.*

Lecture Identification

Lecturer: Philip Dart

Semester 1, 2025

© University of Melbourne 2025

These slides include materials from:

Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development, Third Edition, by Craig Larman, Pearson Education Inc., 2005.